ROTEIRO DE ESTUDOS · SERVIDORES LINUX · MÓDULO 02

Shell, linha de comando e scripts

Bash, arquivos e diretórios, redirecionamento e pipes, busca de texto com grep e expressões regulares, find, links, compactação, editores, transferência de arquivos e scripts com condições e laços

Nível	Básico
Carga sugerida	2 semanas (16–20 h)
Pré-requisitos	Módulo 01; rhel01 e ubu01 acessíveis por SSH
Relação com RHCSA / LFCS	RHCSA: "Understand and use essential tools" e "Create simple shell scripts"; LFCS: Essential Commands (20%)

Objetivos do módulo

- Usar o Bash com eficiência: histórico, atalhos, variáveis, expansão e aspas.
- Manipular arquivos e diretórios, inclusive links simbólicos e físicos.
- Redirecionar entrada, saída e erros, e combinar comandos com pipes.
- Buscar texto com `grep` e expressões regulares; recortar e transformar com `cut`, `sort`, `uniq`, `sed` e `awk`.
- Localizar arquivos com `find`.
- Compactar e arquivar com `tar`, `gzip`, `bzip2` e `xz`.
- Editar arquivos com `vim` e `nano`.
- Transferir arquivos com `scp`, `sftp` e `rsync`.
- Escrever scripts com argumentos, condições, laços e captura de saída.

1. O shell

O **shell** interpreta o que você digita. O padrão nas duas distribuições é o **Bash**. O prompt mostra usuário, máquina e diretório; `$` indica usuário comum e `#` indica root.

Atalho	Ação
Tab / Tab Tab	Completa comandos e caminhos / mostra opções
Ctrl+R	Busca no histórico
Ctrl+C	Interrompe o comando em execução
Ctrl+D	Fim de entrada (sai do shell se a linha estiver vazia)
Ctrl+A / Ctrl+E	Início / fim da linha
Ctrl+L	Limpa a tela
`!!` / `!$`	Repete o último comando / último argumento do comando anterior

```
$ history | tail -20
$ echo $SHELL $HOME $USER $PATH
$ type ls cd grep          # comando externo, embutido (builtin) ou alias?
$ alias ll='ls -lah'       # vale para a sessão; para manter, coloque em ~/.bashrc
```

1.1 Variáveis, aspas e expansões

```
$ NOME="servidor web"
$ echo "Olá, $NOME"         # aspas duplas: expande variáveis
$ echo 'Olá, $NOME'         # aspas simples: texto literal
$ echo "Kernel: $(uname -r)"   # substituição de comando
$ echo arquivo{1..3}.log    # expansão de chaves: arquivo1.log arquivo2.log arquivo3.log
$ ls /var/log/*.log         # curinga (glob)
$ echo $((2*1024))          # aritmética
$ export EDITOR=vim         # exporta para processos filhos
```

2. Arquivos e diretórios

```
$ pwd; cd /etc; cd -; cd ~
$ ls -l /etc/ssh            # -l longo, -a ocultos, -h tamanhos legíveis, -t por data, -R
    ↪ recursivo
$ mkdir -p projetos/web/logs
$ touch nota.txt
$ cp -a origem/ destino/     # -a preserva permissões, donos e datas (recursivo)
$ mv antigo.txt novo.txt
$ rm -r pasta/               # cuidado: não há lixeira
$ cat, less, head -n 20, tail -n 50, tail -f /var/log/messages
$ file /usr/bin/ls; stat /etc/passwd; du -sh /var/log; wc -l /etc/passwd
```

2.1 Links

Tipo	Comando	Características
Físico (hard link)	`ln arquivo link`	Outro nome para o mesmo inode; não funciona entre sistemas de arquivos nem para diretórios; apagar um nome não apaga os dados enquanto houver outro
Simbólico (soft link)	`ln -s alvo link`	Arquivo que aponta para um caminho; funciona entre sistemas de arquivos e para diretórios; quebra se o alvo sumir

```
$ ln -s /var/log/nginx logs-web
$ ls -li                    # -i mostra o número do inode
$ readlink -f logs-web
```

3. Redirecionamento e pipes

Todo processo tem três canais: **0 stdin** (entrada), **1 stdout** (saída) e **2 stderr** (erros).

Sintaxe	Efeito
`cmd > arq`	Saída para arquivo (sobrescreve)
`cmd >> arq`	Saída para arquivo (acrescenta)
`cmd 2> erros.txt`	Só os erros para arquivo
`cmd > tudo.txt 2>&1` ou `cmd &> tudo.txt`	Saída e erros no mesmo arquivo
`cmd 2> /dev/null`	Descarta os erros
`cmd < arq`	Arquivo como entrada
`cmd1 | cmd2`	Saída de cmd1 vira entrada de cmd2
`cmd | tee arq`	Mostra na tela e grava no arquivo

```
$ find / -name "*.conf" 2>/dev/null | wc -l
$ dnf list installed | tee pacotes.txt | grep -i ssh
$ cat <<'EOF' > /tmp/exemplo.txt      # "here document": várias linhas de uma vez
linha 1
linha 2
EOF
```

4. Buscar e transformar texto

4.1 grep e expressões regulares

```
$ grep -i "error" /var/log/messages          # -i ignora maiúsculas
$ grep -r "PermitRootLogin" /etc/ssh/         # -r recursivo
$ grep -v "^#" /etc/ssh/sshd_config | grep -v "^$"   # tira comentários e linhas vazias
$ grep -n -c -l -w                            # número da linha, contagem, só nomes, palavra
    ↪ inteira
$ grep -E "^(root|admin):" /etc/passwd        # -E: expressões regulares estendidas
```

Expressão	Significado
`^` / `$`	Início / fim da linha
`.`	Qualquer caractere
`*` / `+` / `?`	Zero ou mais / um ou mais (ERE) / zero ou um (ERE)
`[abc]` / `[^abc]` / `[0-9]`	Um dos caracteres / nenhum deles / intervalo
`|` (com `grep -E`)	Alternativa: `grep -E "erro|falha"`
`{n,m}` (ERE)	Repetição entre n e m vezes

4.2 Ferramentas de texto

```
$ cut -d: -f1,7 /etc/passwd                   # campos 1 e 7 separados por ":"
$ sort -t: -k3 -n /etc/passwd | tail -3       # ordena pelo UID
$ awk -F: '$3 >= 1000 {print $1, $6}' /etc/passwd   # usuários comuns e suas pastas
$ journalctl -u sshd --since today | grep "Failed password" |
    awk '{print $(NF-3)}' | sort | uniq -c | sort -rn | head   # IPs com mais falhas de SSH
$ sed -i.bak 's/^#Port 22/Port 22/' /etc/ssh/sshd_config   # substitui no arquivo, guardando
    ↪ backup .bak
$ tr 'a-z' 'A-Z' < nota.txt
$ diff -u antigo.conf novo.conf
```

> **NOTA**
>
> No Ubuntu o serviço do SSH se chama `ssh` (o 26.04 adicionou o alias `sshd.service`); no RHEL, `sshd`. Ajuste o `journalctl -u` conforme a distribuição.

5. Localizar arquivos

```
$ find /etc -name "*.repo"                    # por nome
$ find /var/log -type f -size +50M            # arquivos maiores que 50 MB
$ find /home -user ana -mtime -7              # do usuário ana, alterados nos últimos 7 dias
$ find / -perm -4000 -type f 2>/dev/null      # arquivos com SUID (auditoria de segurança)
$ find /tmp -type f -mtime +10 -delete        # apaga temporários com mais de 10 dias
$ find /srv -name "*.log" -exec gzip {} \;    # executa um comando em cada resultado
$ which ss; whereis ssh                       # onde está o executável
```

6. Arquivar e compactar

```
$ tar -cvf backup.tar /etc                    # cria (c), verboso (v), arquivo (f)
$ tar -czvf etc.tar.gz /etc                   # com gzip (z)
$ tar -cjvf etc.tar.bz2 /etc                  # com bzip2 (j)
$ tar -cJvf etc.tar.xz /etc                   # com xz (J)
$ tar -tvf etc.tar.gz                         # lista o conteúdo
$ tar -xzvf etc.tar.gz -C /tmp/restaurar      # extrai em outro diretório
$ gzip arquivo; gunzip arquivo.gz; bzip2 / bunzip2; xz / unxz
```

7. Editores

O **vim** está em praticamente todo servidor e é o editor esperado na RHCSA. O **nano** é mais simples e vem no Ubuntu por padrão.

vim	Ação
`i`, `a`, `o`	Entrar no modo de inserção (antes, depois, nova linha)
`Esc`	Voltar ao modo normal
`:w` / `:q` / `:wq` / `:q!`	Salvar / sair / salvar e sair / sair sem salvar
`dd` / `yy` / `p`	Apagar linha / copiar linha / colar
`u` / `Ctrl+R`	Desfazer / refazer
`/texto`, `n`	Buscar, próxima ocorrência
`:%s/velho/novo/g`	Substituir em todo o arquivo
`gg` / `G` / `:42`	Início / fim / linha 42

> **DICA**
>
> Rode `vimtutor` (pacote vim) uma vez. São 30 minutos que economizam horas.

8. Transferir arquivos com segurança

```
$ scp relatorio.txt usuario@10.20.20.20:/tmp/
$ scp -r usuario@10.20.20.10:/etc/httpd ./copia-httpd
$ sftp usuario@10.20.20.20                    # put, get, ls, cd, bye
$ rsync -avz --delete /srv/site/ usuario@10.20.20.20:/srv/site/   # sincroniza só o que mudou
```

9. Scripts em Bash

Um script é um arquivo de texto com comandos. A primeira linha (**shebang**) diz qual interpretador usar.

```
#!/bin/bash
# verifica-servicos.sh - verifica serviços passados como argumento
# Uso: ./verifica-servicos.sh sshd chronyd

if [ $# -eq 0 ]; then
    echo "Uso: $0 servico1 [servico2 ...]" >&2
    exit 1
fi

for svc in "$@"; do
    if systemctl is-active --quiet "$svc"; then
        echo "[OK]    $svc está ativo"
    else
        echo "[FALHA] $svc está parado"
    fi
done

DISCO=$(df --output=pcent / | tail -1 | tr -dc '0-9')
if [ "$DISCO" -ge 85 ]; then
    echo "ATENÇÃO: a raiz está com ${DISCO}% de uso"
fi
```

```
$ chmod +x verifica-servicos.sh
$ ./verifica-servicos.sh sshd chronyd
$ bash -n script.sh      # verifica a sintaxe sem executar
$ bash -x script.sh      # executa mostrando cada comando (depuração)
```

Elemento	Exemplo
Argumentos posicionais	`$1`, `$2`, todos: `"$@"`, quantidade: `$#`, nome do script: `$0`
Código de saída do último comando	`$?` (0 = sucesso)
Testes de arquivo	`[ -f arq ]` existe e é arquivo, `[ -d dir ]` diretório, `[ -r arq ]` legível, `[ -x arq ]` executável
Testes numéricos	`-eq`, `-ne`, `-lt`, `-le`, `-gt`, `-ge`
Testes de texto	`[ "$a" = "$b" ]`, `[ -z "$a" ]` vazio, `[ -n "$a" ]` não vazio
Encadear	`cmd1 && cmd2` (só se cmd1 deu certo), `cmd1 || cmd2` (só se falhou)
Laço sobre arquivos	`for f in /var/log/*.log; do ...; done`
Laço sobre linhas	`while read -r linha; do ...; done < lista.txt`
Escolha	`case "$1" in start) ... ;; stop) ... ;; *) ... ;; esac`

SEGURANÇA

Coloque variáveis **sempre entre aspas** (`"$arq"`). Sem aspas, nomes com espaços ou caracteres especiais quebram o script, e entradas maliciosas podem virar argumentos inesperados. Para scripts mais longos, comece com `set -euo pipefail` para parar no primeiro erro.

LABORATÓRIO

Laboratório 2.1: caça aos logs

- Liste os 10 maiores arquivos de `/var/log` em cada servidor.
- Conte quantos usuários têm shell `/bin/bash` em `/etc/passwd` usando `grep` e `wc`, e depois só com `awk`.
- Extraia do journal as tentativas de login SSH falhas e mostre os IPs com contagem (tente errar a senha de propósito antes).
- Crie links físico e simbólico para um arquivo, apague o original e explique o que aconteceu com cada link.

LABORATÓRIO

Laboratório 2.2: scripts

- Escreva `backup-etc.sh` que gera `/srv/backup/etc-AAAA-MM-DD.tar.gz`, mantém só os 7 mais recentes e registra o resultado com `logger`.
- Escreva `novo-usuario.sh` que lê uma lista de nomes de um arquivo e cria os usuários que ainda não existem (use `id` para testar).
- Publique os dois no GitHub com README e exemplos de uso.

Checklist de revisão

- Qual a diferença entre `>`, `>>`, `2>` e `2>&1`?
- Diferencie aspas simples e duplas. O que faz `$(...)`?
- Escreva um `grep` que mostre linhas não comentadas e não vazias de um arquivo de configuração.
- Diferencie link físico e simbólico. Qual funciona entre sistemas de arquivos?
- Como encontrar arquivos com SUID em todo o sistema?
- Como criar e extrair um `.tar.gz` em outro diretório?
- Em um script, como ler o primeiro argumento e testar se um diretório existe?

Referências oficiais

› Manual do GNU Bash
`https://www.gnu.org/software/bash/manual/`

› Manual do GNU Coreutils
`https://www.gnu.org/software/coreutils/manual/`

› Manual do GNU grep
`https://www.gnu.org/software/grep/manual/`

› Manual do GNU tar
`https://www.gnu.org/software/tar/manual/`

› Objetivos do exame RHCSA (EX200)
`https://www.redhat.com/en/services/training/ex200-red-hat-certified-system-administrator-rhcsa-exam`

› LFCS: domínios e competências
`https://training.linuxfoundation.org/certification/linux-foundation-certified-sysadmin-lfcs/`

› Ubuntu 26.04: rust-coreutils e sudo-rs
`https://documentation.ubuntu.com/release-notes/26.04/summary-for-lts-users/`